

1 What should I submit, where should I submit and by when?

Your submission for this assignment will be one PDF (.pdf) file and one ZIP (.zip) file. Instructions on how to prepare and submit these files are given below.

Assignment Package:

<https://www.cse.iitk.ac.in/users/purushot/courses/ml/2025-26-s/material/assignments/assn2.zip>

Deadline for all submissions: July 12, 2026, 9:59PM IST

Code Validation Script: <https://colab.research.google.com/drive/1J95rRyD1IPpVQaf1g-gS2YihLjkalsnu?usp=sharing>

Code Submission: <https://forms.gle/WzR4RqVZC2ZdQMxu6>

Report Submission: on Gradescope

There is no provision for “late submission” for this assignment

1.1 How to submit the PDF report file

1. The PDF file must be submitted using Gradescope in the *group submission mode*.
2. Unregistered auditors cannot make submissions to this assignment.
3. Make only one submission per assignment group on Gradescope, not one submission per student. Gradescope allows you to submit in groups - please use this feature to make a group submission.
4. **Ensure that you validate your submission files on Google Colab before making your submission** (validation details below). Submissions that fail to work with our automatic judge since they were not validated will incur penalties.
5. Link all group members in your group submission. If you miss out on a group member while submitting, Gradescope will think that person never submitted anything.
6. You may overwrite your group’s submission as many times as you want before the deadline (submitting again on Gradescope simply overwrites the old submission).
7. Do not submit Microsoft Word or text files. Prepare your report in PDF using the style file we have provided (instructions on formatting given later).

1.2 How to submit the code ZIP file

1. Your ZIP file should contain a single Python (.py) file and nothing else. The reason we are asking you to ZIP that single Python file is so that you can password protect the ZIP file. Doing this safeguards you since even after you upload your ZIP file to your website,

no one can download that ZIP file and see your solution (since you will tell the password only to the instructor). If you upload a naked Python file to your website, someone else may guess the location where you have uploaded your file and steal it and you may get charged with plagiarism later.

2. We do not care what you name your ZIP file but the (single) Python file sitting inside the ZIP file must be named “submit.py”. There should be no sub-directories inside the ZIP file – just a single file. We will look for a single Python (.py) file called “submit.py” inside the ZIP file and delete everything else present inside the ZIP file.
3. Do not submit Jupyter notebooks or files in other languages such as C/C++/R/Julia/Matlab/Java. We will use an automated judge to evaluate your code which will not run code in other formats or other languages (submissions in other languages will get a zero score).
4. Password protect your ZIP file using a password with 8-10 characters. Use only alphanumeric characters (a-z A-Z 0-9) in your password. Do not use special characters, punctuation marks, whitespaces etc in your password. Specify the file name properly in the Google form.
5. Remember, your file is not under attack from hackers with access to supercomputers. This is just an added security measure so that even if someone guesses your submission URL, they cannot see your code immediately. A length 10 alphanumeric password (that does not use dictionary phrases and is generated randomly e.g. 2x4kPh02V9) provides you with more than 55 bits of security. It would take more than 1 million years to go through all $> 2^{55}$ combinations at 1K combinations per second.
6. Make sure that the ZIP file does indeed unzip when used with that password (try `unzip -P your-password file.zip` on Linux platforms).
7. Upload the password protected ZIP file to your IITK (CC or CSE) website (for CC, log on to `webhome.cc.iitk.ac.in`, for CSE, log on to `turing.cse.iitk.ac.in`).
8. Fill in the following Google form to tell us the exact path to the file as well as the password <https://forms.gle/WzR4RqVZC2ZdQMxu6>
9. **Do not host your ZIP submission file on file-sharing services like Dropbox or Google drive. Host it on IITK servers only.** We will autodownload your submissions and GitHub, Dropbox and Google Drive servers often send us an HTML page (instead of your submission) when we try to download your file. Thus, it is best to host your code submission file locally on IITK servers.
10. While filling in the form, you have to provide us with the password to your ZIP file in a designated area. Write just the password in that area. For example, do not write “Password: helloworld” in that area if your password is “helloworld”. Instead, simply write “helloworld” (without the quotes) in that area. Remember that your password should contain only alphabets and numerals, no spaces, special or punctuation characters.
11. While filling the form, give the complete URL to the file, not just to the directory that contains that file. The URL should contain the filename as well.

(a) Example of a proper URL:

`https://web.cse.iitk.ac.in/users/purushot/mlproj/my_submit.zip`

- (b) Example of an improper URL (file name missing):
`https://web.cse.iitk.ac.in/users/purushot/mlproj/`
 - (c) Example of an improper URL (incomplete path):
`https://web.cse.iitk.ac.in/users/purushot/`
12. We will use an automated script to download all your files. If your URL is malformed or incomplete, or if you have hosted the file outside IITK in a manner that is difficult to download, then your group may lose marks.
 13. Make sure you fill-in the Google form with your file link before the deadline. We will close the form at the deadline.
 14. Make sure that your ZIP file is actually available at the link at the time of the deadline. We will run a script to automatically download these files after the deadline is over. If your file is missing, we will treat this as a blank submission.
 15. We will entertain no submissions over email, Piazza etc. All submissions must take place before the stipulated deadline over the Gradescope and the Google form. The PDF file must be submitted on Gradescope at or before the deadline and the ZIP file must be available at the link specified on the Google form at or before the deadline.

Problem 2.1 (Bit-in-the-Middle Attack). While studying for the CS771 endsem exam, Melbo wanted to create a dataset of CRPs for a standard arbiter PUF to study them, each CRP having a 17-bit challenge and a 1-bit response. However, intent on creating mischief, Melbruh secretly distorted the CRPs as well as censored them in the following manner:

1. Before the responses got generated, Melbruh swapped the middle bit of the 17-bit challenge (i.e. the 9-th bit) with some other value. Thus, for certain challenges, the middle bit remained the same and for others, it got flipped. Melbo has no idea for which CRPs was the middle bit flipped.
2. After responses were generated for these 17-bit challenges, Melbruh quietly erased the middle bit so that the challenge now looked like a 16-bit vector instead.

With the exam date approaching, Melbo is worried about solving this problem quickly but correctly recognizes this as a latent-variable learning problem. In this exercise, we will help Melbo learn the PUF model despite Melbruh's malicious meddling. Given a challenge bit-vector $\mathbf{c} \in \{0, 1\}^{16}$ and a (middle) bit value $z \in \{0, 1\}$, we define the *insert operation* as follows:

$$\mathbb{I}(\mathbf{c}, z) \stackrel{\text{def}}{=} [\mathbf{c}[1:8], z, \mathbf{c}[9:]] \in \{0, 1\}^{17},$$

i.e., the operation inserts the bit in the middle position, creating a 17-bit vector. Also, let $\phi : \{0, 1\}^{17} \rightarrow \{-1, +1\}^{17}$ denote the standard embedding/encoding for 17-bit arbiter PUFs as discussed in class. Let $(\mathbf{w}, b) \in \mathbb{R}^{17} \times \mathbb{R}$ denote the 17-dimensional linear model corresponding to the 17-bit PUF. For a CRP $(\mathbf{c}_i, r_i) \in \{0, 1\}^{16} \times \{0, 1\}$ and a value of the hidden bit $z_i \in \{0, 1\}$, Melbo proposes the following likelihood model, where σ is the sigmoid function.

$$\mathbb{P}[r_i \mid z_i, \mathbf{c}_i, \mathbf{w}, b] = \sigma \left((2r_i - 1) \cdot \left(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, z_i)) + b \right) \right),$$

Your Data. We have provided you with data with 16-bit challenges and 1-bit response. The training set consists of 8000 CRPs and the test set consists of 2000 CRPs. For each CRP, a 16-bit challenge and a 1-bit response is provided. Each row in the training and testing files provided to you contain 17 bits, each number being 0 or 1. The first 16 numbers correspond to the challenge bits and the last number corresponds to the response bit. If you wish, you may create (held out/k-fold) validation sets out of this data in any way you like. (150 marks)

Your Task. The following enumerates 5 parts to the question. Parts 1, 3, 5 need to be answered in the PDF report file. Parts 2, 4 need to be answered in the Python file.

1. Melbo wants to learn the MLE PUF model $(\mathbf{w}, b) \in \mathbb{R}^{17} \times \mathbb{R}$. Assuming CRPs to be independent, taking negative logs, and introducing latent variables $z_i \in \{0, 1\}$ for the middle bits introduced by Melbruch gives us the following latent variable MLE formulation:

$$\min_{(\mathbf{w}, b) \in \mathbb{R}^{17} \times \mathbb{R}} - \sum_{i=1}^{8000} \ln \mathbb{P}[r_i \mid \mathbf{c}_i, \mathbf{w}, b] = \min_{(\mathbf{w}, b) \in \mathbb{R}^{17} \times \mathbb{R}} - \sum_{i=1}^{8000} \ln \left(\sum_{z_i \in \{0, 1\}} \mathbb{P}[r_i, z_i \mid \mathbf{c}_i, \mathbf{w}, b] \right)$$

Taking the winner-take-all approximation gives us

$$\min_{\substack{(\mathbf{w}, b) \in \mathbb{R}^{17} \times \mathbb{R}, \\ \{z_i\} \in \{0, 1\}}} - \sum_{i=1}^{8000} \ln \mathbb{P}[r_i, z_i \mid \mathbf{c}_i, \mathbf{w}, b]$$

Assuming a latent prior $\mathbb{P}[z_i \mid \mathbf{c}_i, \mathbf{w}, b] = 0.5$ for all $i \in [8000]$, $z_i \in \{0, 1\}$, $\mathbf{w} \in \mathbb{R}^{17}$, $b \in \mathbb{R}$ and using Melbo's likelihood model, derive an alternating optimization algorithm to learn the model $(\mathbf{w}, b)$ and values of the latent variables $\{z_i\}$, $i \in [8000]$. (20 marks)

2. Write code to implement your alternating optimization algorithm. Submit your code in `submit.py` by implementing a method called `my_latent()` that should take in the 8000 training CRPs and return three objects, a model $\mathbf{w} \in \mathbb{R}^{17}$, a bias term $b \in \mathbb{R}$, and 8000 Boolean values corresponding to the middle bits $z_i \in \{0, 1\}$, $i \in [8000]$. Your returned values will be evaluated based on how well they predict the train responses. (40 marks)
3. To verify whether the values returned by the above alternating optimization algorithm are correct or not, irrespective of whether they result in correct prediction of responses, Melbo and Melba caught hold of the culprit. Under intense interrogation, Melbruch revealed that the SSD containing the middle bit values has been lost, much to Melbo's dismay. However, Melba managed to extract a valuable peice of information that the middle bits introduced by Melbruch were not random or arbitrary – they were, in fact, responses of the remaining 16 bits $\mathbf{c}_i$ on a hidden 16-bit arbiter PUF. Given this, Melbo immediately proposed an updated set of likelihood and latent prior distributions by letting $(\mathbf{u}, a) \in \mathbb{R}^{16} \times \mathbb{R}$ denote the model corresponding to the hidden 16-bit PUF, and abusing notation to let ϕ denote the standard embedding/encoding for 16-bit arbiter PUFs as well:

$$\begin{aligned} \mathbb{P}[r_i \mid z_i, \mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a] &= \sigma \left((2r_i - 1) \cdot \left(\mathbf{w}^\top \phi(\mathbb{I}(\mathbf{c}_i, z_i)) + b \right) \right) \\ \mathbb{P}[z_i \mid \mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a] &= \sigma \left((2z_i - 1) \cdot \left(\mathbf{u}^\top \phi(\mathbf{c}_i) + a \right) \right) \end{aligned}$$

Give derivation for an alternating optimization algorithm to solve the following MLE problem by introducing latent variables, applying the updated likelihood and latent prior distributions, and using the winner-take-all approximation:

$$\max_{\substack{(\mathbf{w}, b) \in \mathbb{R}^{17} \times \mathbb{R} \\ (\mathbf{u}, a) \in \mathbb{R}^{16} \times \mathbb{R}}} \prod_{i=1}^{8000} \mathbb{P}[r_i \mid \mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a]$$

Note that now there are two PUFs involved, a 17-bit one and a 16-bit one. (30 marks)

4. Write code to implement this updated alternating optimization algorithm. Submit your code in (the same) `submit.py` by implementing a method called `my_latent_updated()` that should take in the 8000 training CRPs and return four objects, $\mathbf{w} \in \mathbb{R}^{17}, b \in \mathbb{R}, \mathbf{u} \in \mathbb{R}^{16}, a \in \mathbb{R}$. Note that this updated method should not return values of latent variables since it is returning a model to predict those latent variables. Your returned values will be evaluated based on how well they predict the test responses. (40 marks)
5. Did you find the two algorithms `my_latent()` and `my_latent_updated()` to agree on their outputs? Specifically, suppose `my_latent()` returns $\mathbf{w}, b, \{z_i\}$ and `my_latent_updated()` returns $\tilde{\mathbf{w}}, \tilde{b}, \tilde{\mathbf{u}}, \tilde{a}$, then
 - (a) Did you observe $\mathbf{w} \approx \tilde{\mathbf{w}}$?
 - (b) Did you observe $b \approx \tilde{b}$?
 - (c) For what fraction of the 8000 train CRPs did you observe $2 \cdot z_i - 1 = \text{sign}(\tilde{\mathbf{u}}^\top \phi(\mathbf{c}_i) + \tilde{a})$?

Explain your observations – why do you think the two algorithms agreed/disagreed on their outputs? (20 marks)

Note that both methods `my_latent()` and `my_latent_updated()` must be provided in the same `submit.py` file and not in separate files.

Evaluation Measures and Marking Scheme We created two sets of PUFs – a public one and a private one. Each set contained a 17-bit arbiter PUF and a 16-bit arbiter PUF. Then we created two accompanying sets of train-test CRPs. Both contain 16-bit challenges and 1-bit responses. For the public train-test sets that we have provided you, the middle bit was generated using responses from the 16-bit PUF and the final responses were generated after inserting the middle bit and obtaining responses from the 17-bit PUF. The two sets of PUFs are distinct. Thus, models $\mathbf{c}_i, \mathbf{w}, b, \mathbf{u}, a$ that can predict the train/test responses in the public set provided to you are expected to do poorly at predicting the responses for the private train/test responses and vice versa.

We will use your `my_latent()` method to obtain the 17-dimensional model and middle bit values using which we will attempt to predict the train responses. Note that test responses will not be evaluated for `my_latent()` since we have not obtained test latent variable values from the method. Subsequently, we will use your `my_latent_updated()` method to obtain two models, a 16-dimensional one and a 17-dimensional one. We will use the 16-dimensional model to predict the middle bits for test CRPs and then apply the 17-dimensional model to get test response predictions. Note that test responses can be evaluated for `my_latent_updated()` since we now have a method to predict the middle bits and the middle bits are no-longer free/independent latent variables. All evaluations will be done on the private train/test sets. Note that the private train/test set may not have the same number of train/test points as the public train/test set that we have provided you. However, the public and private CRPs were generated in a similar manner so it is likely that hyperparameter choices that seem good to you during validation (e.g. C , tol, solver, number of alternations etc), should work decently on the private dataset as well. **Please see the validation code for clarification and detailed description of evaluation.**

Important Note: The optimization problems in this assignment are non-convex problems which may be sensitive to initialization. For example, you may have to initialize the latent variables randomly but some initializations may not work so you may have to try again and again. You may yourself run multiple initializations within your code until you get a favorable initialization (confirmed by excellent performance on train/held-out validation set). However,

to help you further, the repetition policy will be a relaxed one for this assignment. Usually, we repeat the evaluation process several times and use the average performance. However, for this assignment, we will **increase the number of repetitions** from the usual 5 and take the **best performance across repetitions instead of the average performance** to award marks. This will further guard submissions against any unluckiness due to random choices inside your code in a particular trial. We will award marks based on the following performance parameters:

1. Time taken by your `my_latent()` method to generate model and latent variables. These marks will be scaled by a train performance multiplier (details below). (40 marks)
2. Time taken by your `my_latent_updated()` method to generate the two models. These marks will be scaled by a test performance multiplier (details below). (40 marks)

Once we receive your code, we will execute the validation script to award marks to your submission. For all the above performance parameters, lower is better. For more details, please check the validation script on Google Colab (linked below). The marks received by the `my_latent()` method will be multiplied by a train performance multiplier whereas marks received by the `my_latent_updated()` method will be multiplied by a test performance multiplier. These multipliers η will be some function of train/test accuracy. For example, the multiplier could be $\eta = 10 \cdot \max(0, a - 0.9)$ where a is the train/test accuracy on the private dataset. The exact formula for the multiplier will be announced along with the graded copies. The reason for the introduction of these multipliers is to penalize trivial submissions (e.g. null models and constant latent variables) that run super fast but give poor accuracy.

Validation on Google Colab. Before making a submission, you must validate your submission on Google Colab using the script linked below.

Link: <https://colab.research.google.com/drive/1J95rRyD1IPpVQaf1g-gS2YihLjkalsnu?usp=sharing>

Validation ensures that your submitted file `submit.py` does work with the automatic judge and does not give errors. Please use the IPYNB file at the above link on Google Colab and the dummy private train, dummy private test set and dummy model files (details below) to validate your submission.

Please make sure you do this validation on Google Colab itself. **Do not download the IPYNB file and execute it on your machine – instead, execute it on Google Colab itself.** This is because most errors we encounter are due to non-standard library versions on students' personal machines. Thus, running the IPYNB file on your personal machine defeats the whole purpose of validation. You must ensure that your submission runs on Google Colab to detect any library conflict. **Please note that there will be penalties for submissions which were not validated on Google Colab and which subsequently give errors with our automated judge.**

Dummy Submission File and Dummy Private Files. In order to help you understand how we will evaluate your submission using the evaluation script, we have included a dummy private train and private test set in the assignment package itself (see the directory called `dummy`). However, note that these are just copies of the public train, test datasets we provided you. The reason for providing the dummy files is to allow you to check whether the evaluation script is working properly on Google Colab or not. Note that the private files on which we actually evaluate your submission will be different. We have also included a dummy submission file `dummy_submit.py` to show you how your code must be written with `my_latent()` and `my_latent_updated()` functions. Note that the choices made in `dummy_submit.py` are bad

and give poor results. However, this is okay since its purpose is only to show you the code format.

Using Internet Resources. You are allowed to refer to textbooks, internet sources, research papers to find out more about this problem and for specific derivations e.g. the arbiter-PUF problem. However, if you do use any such resource, cite it in your PDF file. There is no penalty for using external resources with attribution but claiming someone else's work (e.g. a book or a research paper) as one's own work without crediting the original author will attract penalties.

Restrictions on Code Usage. You are allowed to use the numpy module in its entirety (**you may need to use `numpy.random` anyway for random initialization**) and any sklearn submodule that learns linear models and helper submodules e.g. calculate accuracy of a learnt model using `sklearn.metrics.accuracy_score`. However, **the use of non-linear models is prohibited** e.g. decision trees, random forests, nearest neighbors, neural networks, kernel SVMs etc. The use of other machine learning libraries such as **scipy, libsvm, keras, tensorflow is also prohibited**. Use of prohibited modules and libraries for whatever reason will result in penalties. For this assignment, you should also not download any code available online or use code written by persons outside your assignment group. Direct copying of code from online sources or amongst assignment groups will be considered an act of plagiarism for this assignment and penalized according to pre-announced policies.

2 How to Prepare the PDF File

Use the following style file to prepare your report. The ZIP file contains an example file and instructions.

<https://media.neurips.cc/Conferences/NeurIPS2025/Styles.zip>

You must use the following command in the preamble

```
\usepackage[preprint]{neurips_2025}
```

instead of `\usepackage[preprint]{neurips_2025}`. Use proper L^AT_EX commands to neatly typeset your responses to the various parts of the problem. Use neat math expressions to typeset your derivations. Remember that all parts of the question need to be answered in the PDF file. All plots must be generated electronically - hand-drawn plots are unacceptable. All plots must have axes titles and a legend indicating what are the plotted quantities. Insert the plot into the PDF file using L^AT_EX `\includegraphics` commands.

3 How to Prepare the Python File

The assignment package contains a skeleton file `submit_template.py` which you should fill in with the code of the method you think works best among the methods you tried. You must use this skeleton file to prepare your Python file submission (i.e. do not start writing code from scratch). This is because we will autograde your submitted code and so your code must have its input output behavior in a fixed format. Be careful not to change the way the skeleton file accepts input and returns output.

1. The skeleton code has comments placed to indicate non-editable regions. **Do not remove those comments.** We know they look ugly but we need them to remain in the code to keep demarcating non-editable regions.

2. The code file you submit should be self contained and should not rely on any other files (e.g. other .py files or else pickled files etc) to work properly. Remember, your ZIP archive should contain only one Python (.py) file. This means that you should store any hyperparameters that you tune (e.g. C etc) inside that one Python file itself using variables/functions.
3. Certain portions of the skeleton code have been marked as non-editable. Please do not change these lines of code. Insert your own code within the designated areas only. If you tamper with non-editable code (for example, to make your code seem better or faster than it really is), the auto-grader may refuse to run your code and give you a zero instead (we will inspect each code file manually).
4. You are allowed to freely define new functions, new variables, new classes in inside your submission Python file while not changing the non-editable code.
5. The use of non-linear models is prohibited e.g. decision trees, random forests, nearest neighbors, neural networks, kernel SVMs etc. The use of other machine learning libraries such as scipy, libsvm, keras, tensorflow is also prohibited.
6. You are allowed to use basic Python libraries such as numpy, sklearn which have already been included for you.
7. Do take care to use broadcasted and array operations as much as possible and not rely on loops to do simple things like take dot products, calculate norms etc otherwise your solution will be slow and you may get less marks.
8. Please do not perform file operations in your code or access the disk – you should not be using libraries such as sys, pickle in your code.
9. Before submitting your code, make sure you validate on Google Colab to confirm that there are no errors etc.
10. You do not have to submit the evaluation script to us – we already have it with us. We have given you access to the Google Colab evaluation script just to show you how we would be evaluating your code and to also allow you to validate your code.